내장 함수 시스템 (Builtin Functions System)

Monkey 언어의 내장 함수 시스템은 언어에 기본적으로 포함된 함수들을 제공하는 메커니즘입니다. 이 시스템을 통해 `len`, `first`, `last`, `rest`, `push`, `puts` 등의 유용한 함수들을 사용할 수 있습니다.

1. builtins 맵과 내장 함수 등록 시스템

1.1 builtins 맵의 구조

내장 함수들은 `evaluator/builtins.go` 파일에서 `builtins` 맵을 통해 관리됩니다:

```
var  builtins = map[string]*object.Builtin{
object.BuiltinFuncNameLen: object.GetBuiltinByName(object.BuiltinFuncNameLen),
object.BuiltinFuncNamePuts: object.GetBuiltinByName(object.BuiltinFuncNamePuts),
object.BuiltinFuncNameFirst: object.GetBuiltinByName(object.BuiltinFuncNameFirst
object.BuiltinFuncNameLast: object.GetBuiltinByName(object.BuiltinFuncNameLast),
object.BuiltinFuncNameRest: object.GetBuiltinByName(object.BuiltinFuncNameRest),
object.BuiltinFuncNamePush: object.GetBuiltinByName(object.BuiltinFuncNamePush),
}
```

이 맵의 구조는 다음과 같습니다:

- **키(Key)**: 함수명을 나타내는 문자열 상수
- **값(Value)**: `*object.Builtin` 타입의 포인터

1.2 내장 함수 상수 정의

내장 함수의 이름들은 `object/builtins.go` 에서 상수로 정의됩니다:

```
const (
BuiltinFuncNameLen = "len"
BuiltinFuncNameFirst = "first"
BuiltinFuncNameLast = "last"
BuiltinFuncNameRest = "rest"
BuiltinFuncNamePush = "push"
BuiltinFuncNamePuts = "puts"
)
```

이렇게 상수로 정의함으로써:

- 함수명의 일관성을 보장합니다
- 오타로 인한 버그를 방지합니다
- 코드의 가독성을 향상시킵니다

1.3 Builtin 객체 타입과 함수 래핑

Builtin 구조체 정의

`object/object.go` 에서 `Builtin` 구조체가 정의됩니다:

```
type BuiltinFunction func(args ...Object) Object

type Builtin struct {
```

```
Fn BuiltinFunction

}


func (b *Builtin) Type() ObjectType { return  BuiltinObj }

func (b *Builtin) Inspect() string { return  "builtin function" }
```

함수 래핑 과정

내장 함수들은 `object/builtins.go` 의 `Builtins` 슬라이스에서 정의됩니다:

```
var  Builtins = []struct {

Name  string

Builtin *Builtin

}{

{

BuiltinFuncNameLen,

&Builtin{Fn: func(args ...Object) Object {

if  len(args) != 1 {

return  newError("wrong number of arguments. got=%d, want=1",

len(args))

}


switch  arg := args[0].(type) {

case *Array:

return &Integer{Value: int64(len(arg.Elements))}
```

```
case *String:
return &Integer{Value: int64(len(arg.Value))}
default:
return newError("argument to %q not supported, got %s",
BuiltinFuncNameLen, args[0].Type())
}
}},
},
// 다른 내장 함수들...
}
```

각 내장 함수는 다음과 같은 구조로 래핑됩니다:

1. **함수명**: 문자열로 된 함수 이름
2. **Builtin 객체**: 실제 함수 로직을 포함하는 `Builtin` 구조체

1.4 GetBuiltinByName 함수

내장 함수를 이름으로 검색하는 `GetBuiltinByName` 함수:

```
func GetBuiltinByName(name string) *Builtin {
for _, def := range Builtins {
if def.Name == name {
return def.Builtin
}
}
```

```
return nil
}
```

이 함수는:

- `Builtins` 슬라이스를 순회하며 이름이 일치하는 함수를 찾습니다
- 찾으면 해당 `Builtin` 객체를 반환합니다
- 찾지 못하면 `nil` 을 반환합니다

2. 내장 함수 호출과 매개변수 처리

2.1 내장 함수 식별과 호출 과정

식별자 평가에서의 내장 함수 검색

`evaluator/evaluator.go` 의 `evalIdentifier` 함수에서 내장 함수를 검색합니다:

```
func evalIdentifier(node *ast.Identifier, env *object.Environment) object.Objec
// 1. 먼저 환경(Environment)에서 변수를 찾습니다
if val, ok := env.Get(node.Value); ok {
return val
}

// 2. 환경에 없으면 내장 함수에서 찾습니다
if builtin, ok := builtins[node.Value]; ok {
return builtin
}
```

```
// 3. 둘 다 없으면 에러를 반환합니다
return newError("%s: %s", identifierNotFoundError, node.Value)
}
```

이 과정에서 중요한 점은:

- **우선순위**: 사용자 정의 변수가 내장 함수보다 우선합니다
- **검색 순서**: Environment → builtins 맵 → 에러
- **반환값**: 내장 함수가 발견되면 `*object.Builtin` 객체를 반환합니다

함수 호출 처리

`applyFunction` 함수에서 내장 함수 호출을 처리합니다:

```
func applyFunction(fn object.Object, args []object.Object) object.Object {
switch fn := fn.(type) {
case *object.Function:
// 사용자 정의 함수 처리
extendedEnv := extendedFunctionEnv(fn, args)
evaluated := Eval(fn.Body, extendedEnv)
return unwrapReturnValue(evaluated)
case *object.Builtin:
// 내장 함수 처리
if result := fn.Fn(args...); result != nil {
return result
```

```
} else {
return  Null
}
default:
return  newError("%s: %s", notFunctionError, fn.Type())
}
}
```

2.2 매개변수 전달 방식

가변 인자 처리

내장 함수는 `func(args ...Object) Object` 시그니처를 사용합니다:

```
type BuiltinFunction func(args ...Object) Object
```

이를 통해:

- **유연한 인자 개수**: 함수마다 다른 개수의 인자를 받을 수 있습니다
- **타입 안전성**: 모든 인자는 `Object` 인터페이스를 구현해야 합니다
- **런타임 검증**: 함수 내부에서 인자 개수와 타입을 검증합니다

매개변수 검증 예시

`len` 함수의 매개변수 검증:

```
&Builtin{Fn: func(args ...Object) Object {
// 1. 인자 개수 검증
if  len(args) != 1 {
```

```go
return newError("wrong number of arguments. got=%d, want=1",
len(args))
}

// 2. 인자 타입 검증 및 처리
switch arg := args[0].(type) {
case *Array:
return &Integer{Value: int64(len(arg.Elements))}
case *String:
return &Integer{Value: int64(len(arg.Value))}
default:
return newError("argument to %q not supported, got %s",
BuiltinFuncNameLen, args[0].Type())
}
}}
```

2.3 반환값 처리와 에러 핸들링

정상 반환값 처리

내장 함수는 다양한 타입의 객체를 반환할 수 있습니다:

```go
// Integer 반환 (len 함수)
return &Integer{Value: int64(len(arg.Elements))}
```

```go
// Array 반환 (rest 함수)
return &Array{Elements: newElements}


// nil 반환 (빈 배열 처리)
return nil
```

에러 처리 메커니즘

내장 함수에서 에러가 발생하면 `Error` 객체를 반환합니다:

```go
func newError(format string, a ...interface{}) *Error {
return &Error{Message: fmt.Sprintf(format, a...)}
}


// 사용 예시
return newError("wrong number of arguments. got=%d, want=1", len(args))
return newError("argument to %q not supported, got %s",
BuiltinFuncNameLen, args[0].Type())
```

nil 반환값 처리

`applyFunction` 에서 nil 반환값을 `Null` 객체로 변환합니다:

```go
case *object.Builtin:
if result := fn.Fn(args...); result != nil {
```

```
return result
} else {
return Null // nil을 Null 객체로 변환
}
```

2.4 내장 함수별 매개변수 처리 예시

puts 함수 - 가변 인자 처리

```
{
BuiltinFuncNamePuts,
&Builtin{Fn: func(args ...Object) Object {
// 모든 인자를 출력 (인자 개수 제한 없음)
for _, arg := range args {
fmt.Println(arg.Inspect())
}
return nil
}},
}
```

push 함수 - 정확한 인자 개수 요구

```
{
BuiltinFuncNamePush,
&Builtin{Fn: func(args ...Object) Object {
```

```
// 정확히 2개의 인자 필요
if  len(args) != 2 {
return  newError("wrong number of arguments. got=%d, want=2",
len(args))
}
// 첫 번째 인자는 반드시 Array 타입
if  args[0].Type() != ArrayObj {
return  newError("argument to %q must be %s, got %s",
BuiltinFuncNamePush, ArrayObj, args[0].Type())
}

// 배열에 새 요소 추가
arr := args[0].(*Array)
length := len(arr.Elements)
newElements := make([]Object, length+1, length+1)
copy(newElements, arr.Elements)
newElements[length] = args[1]

return &Array{Elements: newElements}
}},
}
```## 3
. 새로운 내장 함수 추가 방법과 확장 가이드

3.1 새로운 내장 함수 구현 단계

새로운 내장 함수를 추가하려면 다음 단계를 따라야 합니다:

단계 1: 함수명 상수 정의

`object/builtins.go`에 새로운 함수명 상수를 추가합니다:

```go
const (
BuiltinFuncNameLen = "len"
BuiltinFuncNamePuts = "puts"
// ... 기존 함수들
BuiltinFuncNameMax = "max"  // 새로운 함수 추가
BuiltinFuncNameMin = "min"  // 새로운 함수 추가
)
```

단계 2: Builtins 슬라이스에 함수 구현 추가

`object/builtins.go` 의 `Builtins` 슬라이스에 새로운 함수를 추가합니다:

```go
var  Builtins = []struct {
Name  string
```

```
Builtin *Builtin

}{

// ... 기존 함수들

{

BuiltinFuncNameMax,

&Builtin{Fn: func(args ...Object) Object {

if len(args) == 0 {

return newError("wrong number of arguments. got=%d, want at least 1",

len(args))

}



var max int64

for i, arg := range args {

integer, ok := arg.(*Integer)

if !ok {

return newError("argument %d to %q must be %s, got %s",

i, BuiltinFuncNameMax, IntegerObj, arg.Type())

}

if i == 0 || integer.Value > max {

max = integer.Value

}

}
```

```go
return &Integer{Value: max}
}},
},
}
```

단계 3: evaluator의 builtins 맵에 등록

`evaluator/builtins.go` 의 `builtins` 맵에 새로운 함수를 추가합니다:

```go
var builtins = map[string]*object.Builtin{
object.BuiltinFuncNameLen: object.GetBuiltinByName(object.BuiltinFuncNameLen),
// ... 기존 함수들
object.BuiltinFuncNameMax: object.GetBuiltinByName(object.BuiltinFuncNameMax),
object.BuiltinFuncNameMin: object.GetBuiltinByName(object.BuiltinFuncNameMin),
}
```

단계 4: 테스트 작성

`evaluator/evaluator_test.go` 의 `TestBuiltinFunctions` 에 테스트 케이스를 추가합니다:

```go
func TestBuiltinFunctions(t *testing.T) {
tests := []struct {
input string
expected interface{}
}{
// ... 기존 테스트들
```

```
{`max(1, 2, 3)`, 3},
{`max(5, 1, 9, 2)`, 9},
{`max(-1, -5, -2)`, -1},
{`max()`, "wrong number of arguments. got=0, want at least 1"},
{`max(1, "hello")`, fmt.Sprintf("argument 1 to %q must be %s, got %s",
object.BuiltinFuncNameMax, object.IntegerObj, object.StringObj)},
}
// ... 테스트 실행 코드
}
```

3.2 타입 검사와 에러 처리 방법

타입 검사 패턴

내장 함수에서 타입 검사를 수행하는 일반적인 패턴들:

```
// 1. 단일 타입 검사
func checkSingleType(arg Object, expectedType ObjectType, funcName string) *Er
if arg.Type() != expectedType {
return newError("argument to %q must be %s, got %s",
funcName, expectedType, arg.Type())
}
return nil
}
```

```go
// 2. 다중 타입 허용
switch arg := args[0].(type) {
case *Array:
// Array 처리
case *String:
// String 처리
default:
return newError("argument to %q not supported, got %s",
funcName, args[0].Type())
}


// 3. 모든 인자가 같은 타입인지 검사
func checkAllSameType(args []Object, expectedType ObjectType, funcName string)
for i, arg := range args {
if arg.Type() != expectedType {
return newError("argument %d to %q must be %s, got %s",
i, funcName, expectedType, arg.Type())
}
}
return nil
}
```

에러 메시지 가이드라인

명확하고 일관된 에러 메시지를 작성하기 위한 가이드라인:

```
// 1. 인자 개수 에러
return  newError("wrong number of arguments. got=%d, want=%d",
len(args), expectedCount)

// 2. 타입 에러 (단일 인자)
return  newError("argument to %q must be %s, got %s",
funcName, expectedType, actualType)

// 3. 타입 에러 (다중 인자)
return  newError("argument %d to %q must be %s, got %s",
argIndex, funcName, expectedType, actualType)

// 4. 지원하지 않는 연산
return  newError("argument to %q not supported, got %s",
funcName, actualType)

// 5. 범위 에러
return  newError("index out of range: %d", index)
```

3.3 언어 확장을 위한 가이드라인

함수 설계 원칙

1. **순수 함수 지향**: 가능한 한 부작용이 없는 순수 함수로 설계합니다
2. **불변성 유지**: 입력 객체를 변경하지 않고 새로운 객체를 반환합니다
3. **일관된 인터페이스**: 기존 내장 함수들과 일관된 패턴을 따릅니다

성능 고려사항

```
// 좋은 예: 효율적인 메모리 사용
func efficientArrayOperation(arr *Array) *Array {
if len(arr.Elements) == 0 {
return arr // 빈 배열은 그대로 반환
}
// 필요한 크기만큼만 할당
newElements := make([]Object, len(arr.Elements)-1)
copy(newElements, arr.Elements[1:])
return &Array{Elements: newElements}
}

// 나쁜 예: 불필요한 메모리 할당
func inefficientArrayOperation(arr *Array) *Array {
// 항상 새로운 배열을 생성 (비효율적)
newElements := make([]Object, 0)
for i := 1; i < len(arr.Elements); i++ {
newElements = append(newElements, arr.Elements[i])
}
```

```go
return &Array{Elements: newElements}
}
```

확장 가능한 아키텍처 패턴

```go
// 1. 함수 그룹별 분리
// math_builtins.go
var  MathBuiltins = []struct {
Name  string
Builtin *Builtin
}{
{BuiltinFuncNameMax, &Builtin{Fn: maxFunction}},
{BuiltinFuncNameMin, &Builtin{Fn: minFunction}},
{BuiltinFuncNameAbs, &Builtin{Fn: absFunction}},
}

// string_builtins.go
var  StringBuiltins = []struct {
Name  string
Builtin *Builtin
}{
{BuiltinFuncNameUpper, &Builtin{Fn: upperFunction}},
{BuiltinFuncNameLower, &Builtin{Fn: lowerFunction}},
}
```

```go
// 2. 동적 등록 시스템
func RegisterBuiltins(builtins []struct{Name string; Builtin *Builtin}) {
for _, builtin := range builtins {
Builtins = append(Builtins, builtin)
}
}
```

컴파일러 지원 고려사항

내장 함수를 컴파일러에서도 지원하려면:

1. **심볼 테이블 등록**: 컴파일러의 심볼 테이블에 내장 함수를 등록해야 합니다
2. **바이트코드 생성**: 내장 함수 호출을 위한 특별한 Opcode가 필요할 수 있습니다
3. **VM 지원**: 가상 머신에서 내장 함수를 실행할 수 있도록 구현해야 합니다

```go
// compiler/symbol_table.go에서 내장 함수 등록 예시
func NewSymbolTable() *SymbolTable {
s := &SymbolTable{}
// 내장 함수들을 심볼 테이블에 등록
for i, builtin := range object.Builtins {
s.DefineBuiltin(i, builtin.Name)
}
return s
```

```
}
```

3.4 실제 확장 예시: 문자열 처리 함수

다음은 문자열 처리를 위한 새로운 내장 함수들을 추가하는 완전한 예시입니다:

```
// object/builtins.go에 추가
const (
// ... 기존 상수들
BuiltinFuncNameUpper = "upper"
BuiltinFuncNameLower = "lower"
BuiltinFuncNameSplit = "split"
)

// Builtins 슬라이스에 추가
{
BuiltinFuncNameUpper,
&Builtin{Fn: func(args ...Object) Object {
if len(args) != 1 {
return newError("wrong number of arguments. got=%d, want=1", len(args))
}
str, ok := args[0].(*String)
if !ok {
return newError("argument to %q must be %s, got %s",
```

```
BuiltinFuncNameUpper, StringObj, args[0].Type())
}
return &String{Value: strings.ToUpper(str.Value)}
}},
},
{
BuiltinFuncNameSplit,
&Builtin{Fn: func(args ...Object) Object {
if  len(args) != 2 {
return  newError("wrong number of arguments. got=%d, want=2", len(args))
}
str, ok := args[0].(*String)
if !ok {
return  newError("first argument to %q must be %s, got %s",
BuiltinFuncNameSplit, StringObj, args[0].Type())
}
sep, ok := args[1].(*String)
if !ok {
return  newError("second argument to %q must be %s, got %s",
BuiltinFuncNameSplit, StringObj, args[1].Type())
}
parts := strings.Split(str.Value, sep.Value)
elements := make([]Object, len(parts))
```

```go
for  i, part := range  parts {
elements[i] = &String{Value: part}
}
return &Array{Elements: elements}
}},
},
```

이러한 확장을 통해 Monkey 언어의 기능을 점진적으로 향상시킬 수 있습니다.